

Програмирование на С

Лекция 6

Функции

Что мы уже умеем?

- выводить данные (`printf`)
- считывать данные (`scanf`)
- выполнять вычисления
- использовать условия
- использовать циклы
- работать с массивами

Но наши программы становятся всё длиннее.

Откуда взялись `printf()` и `scanf()`?

Мы ими пользуемся.

Но мы их не писали.

Их написали разработчики стандартной библиотеки языка `C`.

А можем ли мы написать свою функцию?

Конечно.

Можно создавать собственные функции так же, как мы используем готовые.

Рассмотрим математическую задачу

Найти число сочетаний

$$C_n^k = \frac{n!}{k!(n-k)!}$$

Как хотелось бы написать программу?

```
int combinations =  
    factorial(n) /  
    (factorial(k) * factorial(n - k));
```

Даже не зная реализации функции `factorial()` , легко понять смысл программы.

А если функции нет?

```
int a = 1;
for (int i = 1; i <= n; i++)
    a *= i;

int b = 1;
for (int i = 1; i <= k; i++)
    b *= i;

int c = 1;
for (int i = 1; i <= n - k; i++)
    c *= i;

int combinations = a / (b * c);
```

Код стал значительно длиннее.

Что такое функция?

Функция — это именованный фрагмент программы, который выполняет определённую задачу.

Функцию можно вызывать столько раз, сколько необходимо.

Собственная функция

```
int factorial(int n)
{
    int result = 1;

    for (int i = 1; i <= n; i++)
    {
        result *= i;
    }

    return result;
}
```

Использование функции

```
int c = factorial(5);  
printf("%d\n", c);
```

При вызове функции управление временно передаётся ей.

После завершения управление возвращается обратно.

Как работает вызов функции?



Что означает `return`?

```
return result;
```

Оператор `return`

- завершает выполнение функции;
- возвращает результат вызова.

Теперь формула выглядит естественно

```
int combinations =  
    factorial(n) /  
    (factorial(k) * factorial(n - k));
```

Программа практически повторяет математическую запись.

Зачем нужны функции?

Функции позволяют

- разделять программу на небольшие части;
- повторно использовать код;
- скрывать детали реализации;
- делать программу более понятной.

Главное

Мы не ограничены готовыми функциями языка.

Как программисты, мы можем создавать собственные функции для решения своих задач.

```
factorial(n)  
max(a, b)  
distance(x1, y1, x2, y2)  
isPrime(n)
```

Именно поэтому большие программы можно делать понятными и удобными для чтения.

Примеры функций

Функция `max`

```
int max(int a, int b) {  
    if (a > b)  
    {  
        return a;  
    }  
    else  
    {  
        return b;  
    }  
}
```


Использование

```
#include <stdio.h>

int main(void) {
    int a, b;
    scanf("%d%d", &a, &b);
    printf("max(%d, %d) = %d", a, b, max(a,b));
    return 0;
}
```